

SCAN Reimplementation with Transformers, Curriculum Learning, and Chain-of-Thought Fine-Tuning

Michael Webster

Abstract

This paper reimplements the original SCAN experiments of [Lake and Baroni \(2018\)](#) using a standard encoder–decoder Transformer. Across all three experiments, our results reproduce the expected behaviour: near-perfect performance on the random split, and clear failures on both the length generalization and primitive-generalization splits. We then extend the analysis by fine-tuning a pretrained T5-small model on SCAN and testing two simple methods aimed at improving length generalization, curriculum learning and input-level chain-of-thought prompting. Both approaches provide small gains on the length split, but neither comes close to solving the task. The results reinforce the main conclusion of the SCAN literature, general-purpose sequence models can recombine familiar patterns but struggle to apply abstract rules in a systematic way.

1 Introduction

The SCAN dataset is designed to test whether sequence-to-sequence models can generalize compositionally rather than rely on pattern matching. The original experiments of [Lake and Baroni \(2018\)](#) show that recurrent networks perform well when train and test commands share surface patterns, but fail on splits that require extrapolating to longer action sequences or applying unseen primitive–modifier combinations. These controlled splits allow us to examine specific dimensions of systematicity: data efficiency (Experiment 1b), length generalization (Experiment 2), and primitive-specific compositionality (Experiment 3).

This paper first reimplements all three SCAN experiments using the standard encoder–decoder Transformer architecture provided in the assignment brief. The goal is to verify the expected baselines and to understand where the model succeeds

and where it breaks down relative to the original RNN results.

The second part of the paper introduces an extension to Experiments 1b and 2. A pretrained T5-small model is fine-tuned on SCAN, and two simple modifications to the training procedure are tested: curriculum learning over target-sequence length, and input-level chain-of-thought prompting. Both methods are motivated by prior work on compositionality and prompting, and the experiments are designed to test whether these techniques can help a small pretrained model overcome the length-generalization failure observed in the baseline Transformer.

2 Reimplementation of SCAN Experiments

This section summarizes the behaviour of our from-scratch encoder–decoder Transformer on SCAN. A comparison of results is made both to the original RNN results of [Lake and Baroni \(2018\)](#) and to the expected Transformer baselines defined in the Assignment 2 brief. Across Experiments 1–3, the picture is consistent. The Transformer matches the near-ceiling performance of the best RNNs on the easy random split, but fails on the length and primitive-generalization splits in a similar way to the recurrent models.

Our implementation follows the provided boilerplate. It includes a standard encoder–decoder Transformer with multi-head self-attention in the encoder, masked self-attention plus cross-attention in the decoder, sinusoidal positional encodings in the encoder, learned positional embeddings in the decoder, and greedy autoregressive decoding at test time. All reported numbers are averaged over five random seeds. Tables report mean $\pm$ standard deviation over runs, as in our group slides.

	Original	Expected	Ours
Seq acc.	99.8%	99.8%	99.8 $\pm$ 0.11%

Table 1: Experiment 1a random split results (sequence-level accuracy).

2.1 Experiment 1: Random Split (1a / 1b)

Experiment 1 uses an 80/20 random split over SCAN commands. In 1a, models are trained on the full 80% training set and evaluated on the remaining 20% test commands. In 1b, the number of distinct training commands is varied (e.g., 1%, 2%, 4%, ...) in order to probe how much combinatorial coverage is needed before generalization saturates.

Lake and Baroni (2018) report that an LSTM without attention (2 layers, 200 hidden units, no dropout) achieves 99.8% sequence-level accuracy on the random split (Table 1). When trained on subsets of the command space, the same architecture quickly approaches ceiling: performance is around 5% at 1% coverage, 54% at 2%, and 93% by 4%, with near-perfect generalization thereafter. This shows that once the model has seen a modest fraction of distinct examples, it can recombine them almost perfectly on held-out commands.

The Assignment 2 brief specifies that for Experiment 1, a 1-layer Transformer with 128-dimensional embeddings, 8 heads, and a 512-dimensional feed-forward layer should reach 99.8% sequence-level accuracy on the main random split. The expected 1b behaviour mirrors the RNN case qualitatively, with accuracy climbing quickly as coverage moves from 1% to a small fraction of the command space and then saturates close to ceiling.

On the full 80/20 random split (1a), our Transformer reproduces the expected baseline almost exactly: 99.8% $\pm$ 0.11% sequence-level accuracy across five runs (Table 1). Once a command’s ingredients are well covered in training, a relatively small Transformer learns the SCAN mapping essentially perfectly.

Although tracking metrics are different between our reimplementation and the original paper for Experiment 1b (we use ordered token accuracy rather than sequence accuracy to match the expected results), the original RNN curve and our Transformer curve show a similar qualitative pattern. Weak performance is observed at 1% coverage, a jump by

2–4%, and near-ceiling accuracy once a small but diverse subset of commands has been seen.

The key point for later analysis is that neither architecture has any problem recombining familiar parts in the random split setting. Both models are efficient with little training data. A small fraction of distinct commands is enough to interpolate to the remaining ones. However, strong systematicity is not proved, as all patterns in the test set have been observed in training.

2.2 Experiment 2: Length Generalization

Experiment 2 isolates length generalization. Training contains all commands whose target action sequences have length ≤ 22 . The test set consists of all remaining commands with action sequences between 24 and 48 actions. Models are evaluated both with standard greedy decoding (“without oracle length”) and under an oracle constraint where decoding is forced to continue until the gold output length is reached (“with oracle length”).

Lake and Baroni (2018) best RNN for this split is a GRU with attention (1 layer, 50 hidden units, dropout 0.5), which reaches 20.8% sequence-level accuracy without length information and 60.2% when given the gold output length at evaluation (Table 2). Even in the oracle setting, accuracy collapses as action sequences get longer. The model generalizes somewhat to the shortest held-out target lengths (24–25 actions), but performance drops sharply beyond that, with length 48 near chance. This length-generalization failure is a core failure that SCAN is designed to expose.

The baseline Transformer almost never predicts the exact correct action sequence end-to-end, although it does produce a majority of the individual actions correctly. Providing the gold output length drastically increases the chance of getting the entire sequence right, but still leaves a large gap to perfect performance (Table 2).

The length-wise plots reinforce the same story as Lake and Baroni (2018). Partial success is almost entirely explained by generalization to the shortest held-out target lengths (see Appendix A) for detailed length curves). Our Transformer almost never produces the entire correct sequence, most commands contain many locally correct actions, and providing the true length helps but does not

	Orig.	Exp.	Ours
Seq.	20.8%	0%	$0.08 \pm 0.16\%$
Tok.	—	66.45%	$65.87 \pm 6.65\%$
Seq. (oracle)	60.2%	54.39%	$53.17 \pm 3.88\%$
Tok. (oracle)	—	88.76%	$86.00 \pm 2.86\%$

Table 2: Experiment 2 length split results, with and without oracle length.

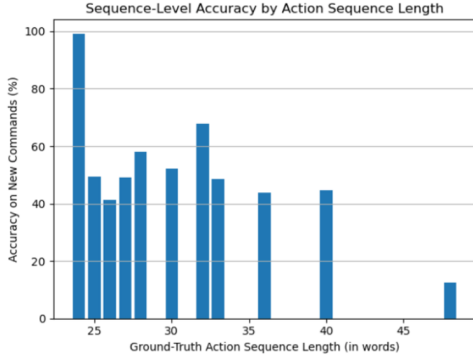


Figure 1: Experiment 2 expected (with oracle) sequence accuracy by action sequence length.

solve the problem, as the model still fails badly on longer sequences.

It is interesting to note that the GRU with attention in the original paper outperforms the transformer model (20.8% v 0% without oracle, 60.2% v 54% with oracle). This gap does not reflect superior compositional reasoning, instead it is likely due to two practical factors. Firstly, the GRU was selected as the top model from an exhaustive hyperparameter search over 180 architectures, whereas the Transformer results come from a single fixed baseline configuration. Secondly, it is plausible the transformer is overfitting to the maximum sequence length in training, while the GRU is less prone to prematurely emit an EOS token under greedy decoding, as token accuracy remains high (65.87% non-oracle) for the transformer. However, once oracle length is enforced, both models still fail on the longer sequences, confirming that neither architecture learns a systematic rule for length generalization consistent with Lake and Baroni (2018) original conclusions.

2.3 Experiment 3: Generalizing Across Primitive Commands

Experiment 3 targets primitive-specific compositionality. The model trains on all primitive and composed commands except compositions involv-

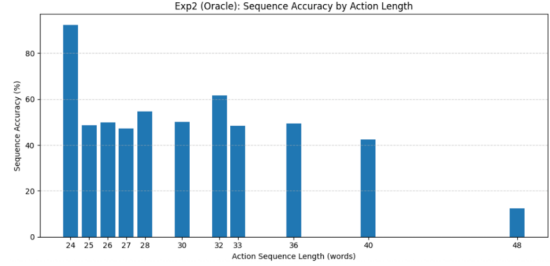


Figure 2: Experiment 2 actual Transformer results (with oracle) sequence accuracy by action sequence length.

	Orig.	Exp.	Ours
Seq. (LTURN)	90.3%	16.39%	$19.09 \pm 8.76\%$
Tok. (LTURN)	—	78.98%	$78.10 \pm 1.29\%$
Seq. (JUMP)	1.2%	0.75%	$0.44 \pm 0.18\%$
Tok. (JUMP)	—	70.06%	$65.38 \pm 4.88\%$

Table 3: Experiment 3 primitive generalisation results (Turn left vs. Jump).

ing one held-out primitive (either “turn left” or “jump”). At test time, it must execute composed commands built from that primitive (e.g., “turn left twice and walk”, “jump around left thrice and run opposite right twice”). The training data still includes all other primitives (walk, run, look, the opposite primitive).

Our Transformer achieved about 19% sequence accuracy on the ‘turn left’ split vs 0.4% on the ‘jump’ split (Table 3), whereas (Lake and Baroni, 2018) RNN got 90% and 1.2% respectively. This deviation for ‘turn left’ is within run-to-run variance (8.8%), so effectively our Transformer replicates the pattern of high performance on the easier primitive, and effectively zero on the hard one.

Findings with the transformer mirror the original in Lake and Baroni (2018). The difference is performance between the “left turn” and “jump” experiments can be explained by a distribution difference. LTURN appears frequently inside other composites during training so the model sees how it composes in context. JUMP appears almost exclusively in the original primitive “jump”. The Transformer baseline fails to achieve anything like the RNN’s 90.3% on “turn left”, but still shows a clear gap between the “easy” primitive (respectable token-level performance, low but non-zero sequence accuracy) and the “hard” primitive (near-zero sequence-level accuracy).

Exp	Seq Acc	Tok Acc
1b	56.99%	97.02%
2	2.03%	68.25%

Table 4: Finetuning on T5-small

The variance between runs can also be seen when composed jump commands are introduced. At 16 composed commands, accuracy varies between runs drastically (from 14.63% up to 70.03%). This indicates that the transformer does not have a stable representation of the “jump” command at this representation level. Small differences in initialization or training order have a large impact on the learned representation. This volatility evens out as more commands are added. This reinforces the finding that the transformer is not learning abstract rules, rather it is reliant on pattern matching over the specific compositions it has observed.

3 New experiment: Curriculum and Chain-of-Thought Fine-Tuning

Both the original RNN and the transformer models have similar shortcomings in each experiment. This section involves exploring the failures of experiment 1b and experiment 2 in more detail.

As observed in the reimplementation and the original, the failings of these two experiments differ. Experiment 1b fails on the small training splits due to a lack of data efficiency. Experiment 2 fails due to a lack of length generalization.

To inspect this more closely, a pretrained transformer model (T5-small) is fine-tuned using the SCAN dataset. On the 1% training split, the fine-tuned T5 model yields 95% token accuracy, compared to 74% from the transformer model earlier. The same fine-tuned model yields 2.3% sequence accuracy and 68.25% token accuracy on the length split, only a slight improvement to the baseline transformer (Table 4). This difference suggests that vanilla pretraining helps data efficiency, but is ineffective for extrapolating to longer sequences.¹

This observation, along with previous research, motivates two new ideas to test on experiment 2:

- **Curriculum learning:** (Liu et al., 2020) provides interesting follow on using the SCAN

dataset. Their approach, using a composer and solver architecture, achieves 100% accuracy on all tasks (including experiment 2). However, when curriculum learning is removed, accuracy drops to near 0. This inspires the proposed methodology here, which focuses on testing curriculum learning when fine-tuning a T5-small model. The model is fine-tuned in stages, beginning with commands whose target action sequences length is ten actions or less. This is then gradually increased to 14, then 18 and finally 22. (Liu et al., 2020) argue that staging the data in this way helps the model tie modifiers to output length. If curriculum is effective, it would suggest that the length failure is partly a training-data issue rather than purely architectural

- **Chain of Thought (CoT) reasoning:** the observed success of chain-of-thought prompting in (Wei et al., 2022) inspires a similar test on experiment two. The proposed method augments the input with a short, deterministic natural-language explanation of how to interpret the command compositionally. This mirrors the style of explanatory prompting used in CoT work, but uses a hard-coded template rather than model-generated thoughts. The model is shown how modifiers such as twice, thrice, around, and after affect the execution procedure, but it must still generate the correct action sequence on its own. The target remains the same SCAN action sequence. The goal is to test whether providing a structured explanation at the input level can help the model internalise the modifier–length relationship, and thereby improve length generalization.

To test their effect on experiment 2, two experiments are conducted using these methods: curriculum only, and curriculum+CoT reasoning. CoT reasoning is chosen as an extension to the curriculum fine-tuning, further research could involve isolating the effect of CoT.

An example of the CoT template used is:

Command: walk twice after run.

Input: translate command to actions: walk twice after run. Reasoning: we decompose the command into simpler sub-commands and modifiers. “after” means we execute “run” first and then “walk twice”.

Target: I_RUN I_WALK I_WALK

¹All results in Section 3 are without oracle length. This is to test true length generalization.

Model	Seq Acc	Tok Acc
T5-small finetune	2.30%	68.25%
T5-small + Curriculum	4.85%	70.92%
T5-small + Cur. + CoT	6.05%	69.55%

Table 5: Additional tests on Experiment 2 (without oracle).

The full-length example used in training is provided in Appendix B

3.1 Results and discussion

The two methods show a modest boost in performance over the baseline on sequence accuracy, showing that both curriculum learning and chain-of-thought reasoning carry merit when using a small fine-tuned model. Sequence accuracy improved from 2.3% to 4.85% with curriculum, and to 6.05% with curriculum+CoT (Table 5), indicating a small absolute gain. Token-level accuracy is on par with the baseline for both. This suggests that incorporating curriculum learning and chain-of-thought into training helped the model arrange sequences more accurately, without dramatically changing the overall proportion of correct tokens. Length-wise graphs (Appendix A) show that improvements are only for sequences of 24 or 25 actions. While both fine-tuning approaches improve over the sequence accuracy of the zero-shot T5 baseline, they do not meaningfully close the length generalization gap.

Notably, these accuracy gains are very modest in absolute terms and fall far short of the 100% generalization performance reported by (Liu et al., 2020) when using curriculum learning. This is likely due to the T5-small transformer lacking the specialized structure for compositional reasoning compared the purpose built neural composer-solver architecture in their experiment, along with the training from scratch regime employed in (Liu et al., 2020), rather than a finetuning approach. This difference highlights the importance of architecture. A model built explicitly to compose and solve sub-tasks can succeed, where an off-the-shelf pretrained model struggles.

The second methods attempt to break down the commands into sub-tasks through chain-of-thought reasoning (CoT) saw a modest increase in accuracy, but it again did not meaningfully close the SCAN length generalization gap. This could be due to how the reasoning was established. Through the

deterministic prompt templates, the model is given extra information about the command’s decomposition, but it does not explicitly train the model’s internal reasoning ability (as the reasoning steps are not generated by the model). The results suggest that this input-level CoT signal was helpful but limited, as it provided some immediate parsing guidance without fundamentally changing the model’s sequence construction strategy. The chain-of-thought cues appeared to help the model get the pieces of the task correct (e.g. recognizing when to perform an “around left” turn), as reflected in the high token accuracy, but they did not reliably ensure the pieces were assembled in the right order or count. The reasoning is always present and not trained, the model may just treat it as extra context tokens, not as steps to be executed.

A core limitation of this work is compute. All experiments rely on local models, which restricts the use of larger architectures.. Prior work (Wei et al., 2022) shows that CoT gains scale with model size. T5-small has around 60M parameters, meaning it lacks both the capacity and the inductive bias needed to exploit CoT effectively. Under these constraints, the evaluation here can only probe whether CoT helps at small scale, not whether it would meaningfully improve length generalization with a suitably large model.

4 Conclusion

Our implementation of the Lake and Baroni (2018) experiments using transformer architecture largely match the expected results. The differences in results from the transformer to the original architectures prove a useful comparison. We attribute differences in results to the internal mechanisms of the architecture, along with the authors model selection process. Simple fine-tuning using a T5-small model provides large performance gains on the 1% split for experiment 1b, showing that fine-tuning a pretrained transformer helps with data efficiency. Both curriculum learning and chain-of-thought reasoning in training provide modest sequence accuracy gains for experiment 2. However, they fail to successfully solve the length generalization task. More careful tuning may be required to improve results. Even with modern pretraining and training techniques, the SCAN length split remains unsolved, underscoring that fundamental architectural or algorithmic innovations are needed for systematic generalization.

References

- Brenden M. Lake and Marco Baroni. 2018. [Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks](#). In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2873–2882. PMLR.
- Qian Liu, Shengnan An, Jian-Guang Lou, Bei Chen, Zeqi Lin, Yan Gao, Bin Zhou, Nanning Zheng, and Dongmei Zhang. 2020. [Compositional generalization by learning analytical expressions](#). In *Advances in Neural Information Processing Systems*, volume 33.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems*.

Appendix A: Additional Length Curves

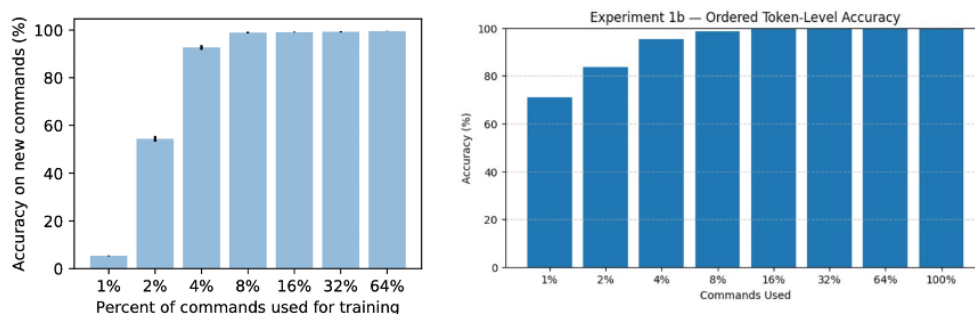


Figure 3: Experiment 1b learning curves (original vs. Transformer).

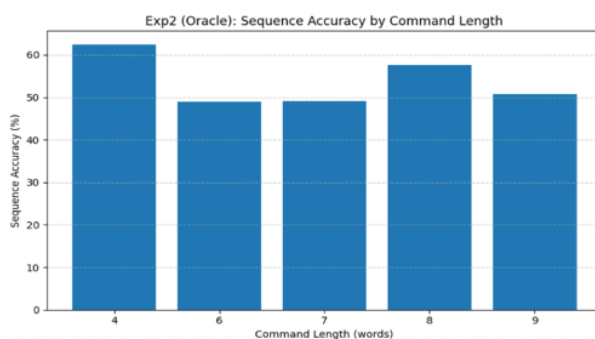


Figure 4: Detailed length-wise curves for Experiment 2.

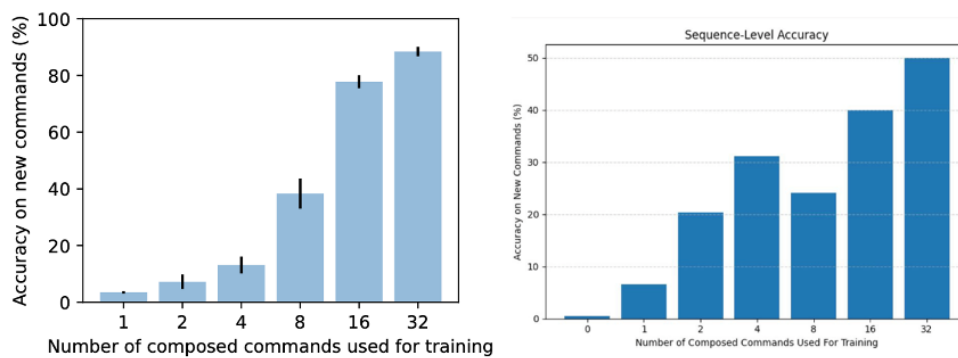


Figure 5: Experiment 3: Jump (original vs. Transformer).

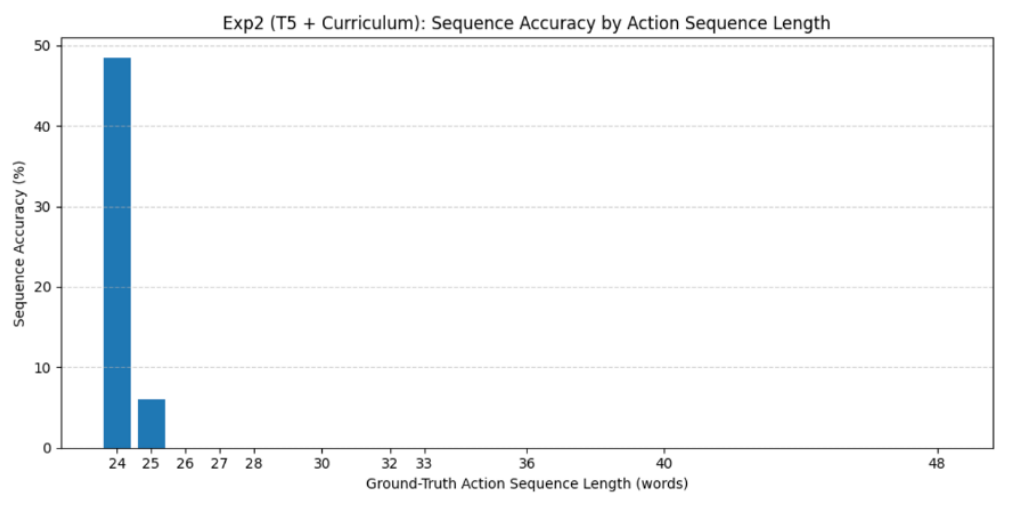


Figure 6: Experiment 2 additional experiments: T5+curriculum action sequence length.

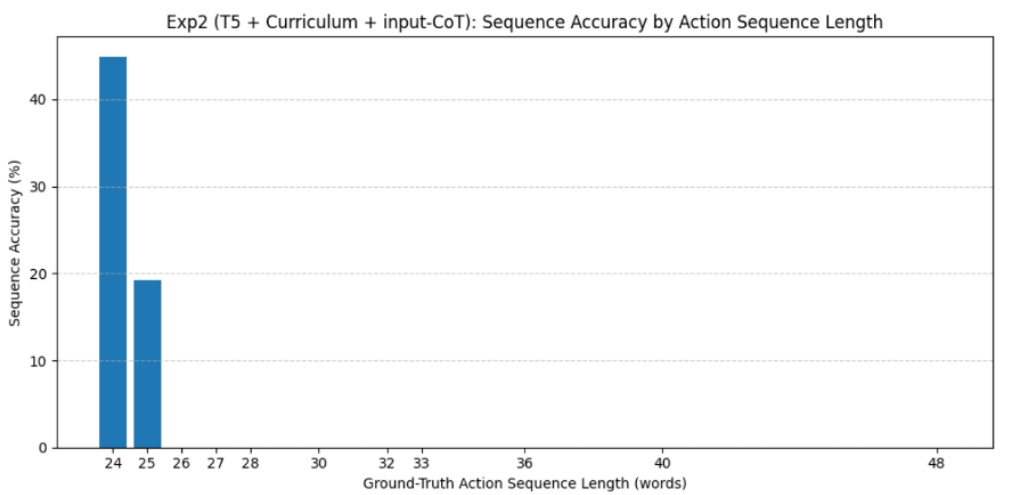


Figure 7: Experiment 2 additional experiments: T5+curriculum+CoT action sequence length.

Appendix B: Chain-of-Thought Example

Command: jump around right twice after look left thrice

Input: translate command to actions: jump around right twice after look left thrice.

Reasoning: we decompose the command into simpler sub-commands and modifiers. “after” means we execute “look left thrice” first and then “jump around right twice”. “around right” means repeatedly turning right and applying the base action in a loop. “twice” means repeating the described movement pattern two times. “thrice” means repeating the described movement pattern three times. The primitive verbs jump, look map directly to their atomic movement actions.

Target: I_TURN_RIGHT I_WALK I_TURN_RIGHT I_WALK I_TURN_RIGHT I_WALK
I_TURN_RIGHT I_WALK I_TURN_RIGHT I_WALK I_TURN_RIGHT I_WALK I_TURN_RIGHT
I_WALK I_TURN_RIGHT I_WALK I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT
I_JUMP

Figure 8: Full chain-of-thought example used in training.

Appendix C: Experimental Setup

All new experiments in Section 3 use the same backbone and optimisation setup. We fine-tune a pretrained T5-small encoder–decoder model with the standard T5 tokenizer. Inputs are prefixed with:

translate command to actions: <command>

Targets are normalised SCAN action sequences (Section 2). Training uses AdamW (learning rate 5×10^{-4}), a linear warm-up and decay schedule (`get_linear_schedule_with_warmup`) with 1% warm-up steps, and gradient clipping at $\|g\|_2 = 1.0$. Mixed-precision (`torch.cuda.amp`) is used on GPU. Hyperparameters for reimplementation follow those outlined in the Assignment 2 brief. For the new experiments, the same hyperparameters are used, except that Experiment 2 runs for 20,000 steps to comply with compute limits. The new experiments are also run once to save compute, therefore limiting variance analysis

Figure 9: Implementation and training configuration used across experiments.

Appendix D: Error Examples

INPUT: translate command to actions: jump around right and jump around left twice
GOLD : I_TURN_RIGHT I_JUMP I_TURN_RIGHT I_JUMP I_TURN_RIGHT I_JUMP
I_TURN_RIGHT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT
I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP
I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP
PRED : I_TURN_RIGHT I_JUMP I_TURN_RIGHT I_JUMP I_TURN_RIGHT I_JUMP
I_TURN_RIGHT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT
I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP I_TURN_LEFT I_JUMP
I_TURN_LEFT I_JUMP

INPUT: translate command to actions: look around left twice after look around left
GOLD : I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK
I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT
I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK
I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK
PRED : I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK
I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT
I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK I_TURN_LEFT I_LOOK
I_TURN_LEFT I_LOOK

Figure 10: Examples of errors in the exp2 T5+Curriculum+CoT. Both examples show a similar pattern to the original experiment 2 failings, where predictions are shorter than gold

Appendix E: AI declaration

ChatGPT (version 5.2) was used for debugging code, proofreading text, and assisting with LaTeX formatting.